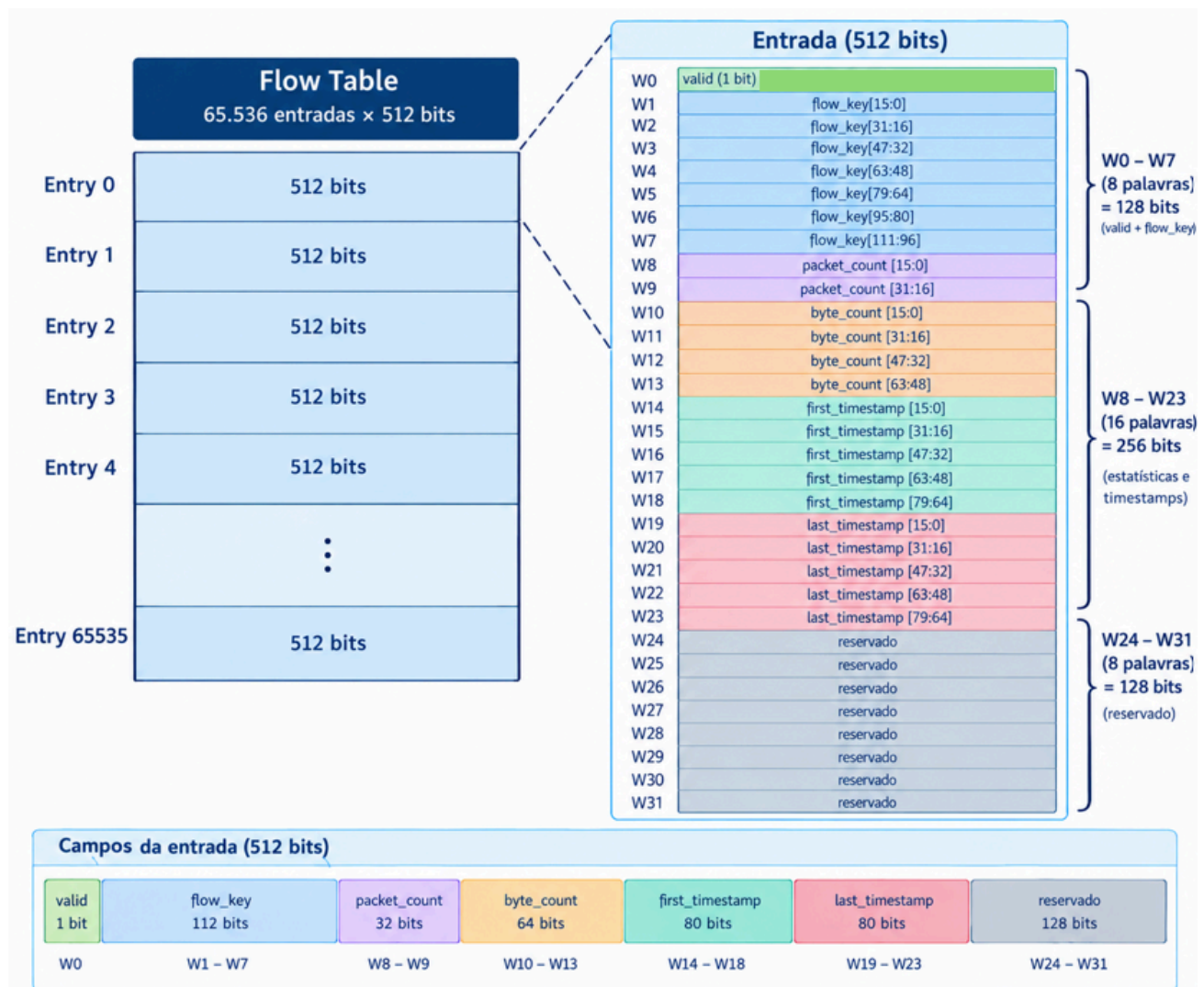


Especificação da Flow Table

A Flow Table é uma memória que armazena o estado dos fluxos identificados pelo sistema. Na arquitetura que definimos, ela possui 65.536 entradas, cada uma associada diretamente a um hash de 16 bits. Cada entrada da Flow Table possui 512 bits, organizados em 32 palavras de 16 bits.

1. Organização geral:



- Entrada lógica (registro de um fluxo completo): 512 bits
- Palavra da memória (configuração física): 16 bits
- Palavras por entrada: 32 (32x16=512)
- Número de entradas = 65536
- Largura do índice hash = 16 bits
- Tamanho total da memória: 65.536x512=33.554.432 bits=4 MiB

Word	Campo	Largura	Função	Capacidade
0	valid	1 bit	Indica se a entrada está ocupada por um fluxo válido.	2 valores (válido/inválido)
1-7	flow_key	104 bits	Identifica o fluxo por meio da 5-tupla	IP origem, IP destino, porta origem, porta destino, protocolo
8-9	packet_count	32 bits	Quantidade de pacotes pertencentes ao fluxo.	Até 4×10^9 pacotes por fluxo.
10-13	byte_count	64 bits	Quantidade de bytes recebidos no fluxo.	Até 6×10^{12} bytes
14-18	first_timestamp	80 bits	Timestamp de chegada do primeiro pacote. 48 bits - seg. 32 bits - nanosseg.	$8,9 \times 10^6$ anos
19-23	last_timestamp	80 bits	Timestamp de chegada do pacote mais recente. 48 bits - seg. 32 bits - nanosseg.	$8,9 \times 10^6$ anos
24-31	Reservado	128 bits	-	-

2. Regras de operação

Para cada novo pacote, o Controller utiliza o hash do fluxo para determinar a entrada a ser consultada: $\text{base_addr} = \text{hash} \ll 5$ (multiplicação por 32), onde base_addr corresponde ao endereço da primeira palavra da entrada, e as 32 palavras subsequentes correspondem ao restante da entrada selecionada.

O processamento de cada pacote é dividido em duas etapas principais: **lookup** e **operação sobre o fluxo**, conforme o resultado da consulta.

a. Lookup

Inicialmente, o Controller realiza a leitura das palavras **W0–W7** da entrada, que contêm:

- W0: valid;
- W1–W7: flow_key.

Os demais campos da entrada não são lidos durante o lookup.

A decisão é realizada da seguinte forma:

- valid = 0 → MISS
- valid = 1 e stored_flow_key = new_flow_key → HIT
- valid = 1 e stored_flow_key ≠ new_flow_key → COLLISION

Portanto, o lookup requer **8 leituras de 16 bits** e não realiza nenhuma escrita na memória.

b. Criação de um novo fluxo — MISS

O Controller cria o novo registro:

```
valid      ← 1
flow_key   ← novo flow_key
packet_count ← 1
byte_count ← packet_length
first_timestamp ← timestamp
last_timestamp ← timestamp
```

A entrada é então gravada na Flow Table. Nesta operação, são escritos os words **W0–W23**, correspondentes aos campos utilizados da entrada:

- W0: valid;
- W1–W7: flow_key;
- W8–W9: packet_count;
- W10–W13: byte_count;
- W14–W18: first_timestamp;
- W19–W23: last_timestamp.

Assim, considerando a implementação inicial, um MISS resulta em:

- **8 leituras** para o lookup;
- **24 escritas** para a criação da entrada.

Os words **W24–W31 permanecem reservados** e não são utilizados nesta operação.

c. Atualização de fluxo existente — HIT

O Controller realiza a leitura dos demais campos necessários para a atualização, correspondentes aos words **W8–W23**. Os campos são atualizados conforme:

valid $\leftarrow 1$

flow_key $\leftarrow$ flow_key

packet_count $\leftarrow$ packet_count + 1

byte_count $\leftarrow$ byte_count + packet_length

first_timestamp $\leftarrow$ first_timestamp

last_timestamp $\leftarrow$ timestamp

Após a atualização, os campos atualizados são escritos novamente na Flow Table. Assim, um HIT resulta em:

- **8 leituras** durante o lookup;
- **16 leituras adicionais** dos words W8–W23;
- **11 escritas** dos words W8-W13 e W19-W23.

d. Colisão — COLLISION

A entrada existente não é modificada e o novo fluxo é descartado.

Assim, uma COLLISION resulta em:

- **8 leituras** para o lookup;
- **0 escritas**.

e. Resumo das operações de memória

Resultado	Lookup	Leituras adicionais	Escritas	Total de leituras
MISS	W0–W7 (8)	0	W0–W23 (24)	8
HIT	W0–W7 (8)	W8–W23 (16)	W8-W13 e W19-W23 (11)	24
COLLISION	W0–W7 (8)	0	0	8

A estratégia de acesso permite que o Controller determine MISS, HIT ou COLLISION sem precisar ler a entrada completa. Somente no caso de HIT são acessados os campos adicionais necessários para atualizar as estatísticas do fluxo.

3. Proposta de interface

Sinal	Direção	Largura	Função
clk	entrada	1	Clock do sistema
rd_en	entrada	1	Solicita uma leitura
rd_addr	entrada	21	Endereço da palavra de 16 bits a ser lida
rd_data	saída	16	Palavra de 16 bits lida da memória
rd_valid	saída	1	Indica que rd_data é válido
wr_en	entrada	1	Solicita uma escrita
wr_addr	entrada	21	Endereço da palavra de 16 bits a ser escrita
wr_data	entrada	16	Palavra de 16 bits a ser escrita
wr_done	saída	1	Indica que a escrita foi concluída

4. Operações

a. Leitura:

A leitura é realizada em duas etapas: solicitação e retorno dos dados.

No ciclo de solicitação, o Controller fornece:

- rd_en = 1
- rd_addr = endereço da palavra

No ciclo seguinte, a Flow Table disponibiliza:

- rd_valid = 1
- rd_data = dado de 16 bits

b. Escrita:

A escrita ocorre quando o Controller fornece:

- wr_en = 1
- wr_addr = endereço da palavra
- wr_data = dado de 16 bits

No ciclo seguinte, a Flow Table atualiza e sinaliza a conclusão:

- memory[wr_addr] <= wr_data
- wr_done = 1

c. Exemplo de um HIT, considerando hash = 5:

Como cada entrada possui **32 words de 16 bits**: base_addr=5×32=160 (endereço da primeira palavra da entrada 5)

Entry 5	
W0	→ endereço 160
W1	→ endereço 161
W2	→ endereço 162
...	
W7	→ endereço 167
W8	→ endereço 168
...	
W23	→ endereço 183
...	
W31	→ endereço 191

- Lookup

O Controller começa lendo W0–W7, pois esses words contêm valid e flow_key.

Ciclos de leitura				
Ciclo	rd_en	rd_addr	Word	Conteúdo
C0	1	160	W0	valid
C1	1	161	W1	flow_key[15:0]
C2	1	162	W2	flow_key[31:16]
C3	1	163	W3	flow_key[47:32]
C4	1	164	W4	flow_key[63:48]
C5	1	165	W5	flow_key[79:64]
C6	1	166	W6	flow_key[95:80]
C7	1	167	W7	flow_key[111:96]

A Flow Table retorna cada palavra através de rd_data e sinaliza rd_valid. Se valid = 1 e stored_flow_key = incoming_flow_key, tem-se um HIT.

- Após o HIT:

O Controller precisa obter os demais dados da entrada para atualizá-los. São lidos **W8–W23**, para saber packet_count, byte_count, first_timestamp, last_timestamp.

Ciclo	Endereço	Word	Campo
C8	168	W8	packet_count
C9	169	W9	packet_count
C10	170	W10	byte_count
C11	171	W11	byte_count
C12	172	W12	byte_count
C13	173	W13	byte_count
C14	174	W14	first_timestamp
C15	175	W15	first_timestamp
C16	176	W16	first_timestamp
C17	177	W17	first_timestamp
C18	178	W18	first_timestamp
C19	179	W19	last_timestamp
C20	180	W20	last_timestamp
C21	181	W21	last_timestamp
C22	182	W22	last_timestamp
C23	183	W23	last_timestamp

- Escrita da Entry 5

Depois dos cálculos de atualização dos estados, o Controller escreve novamente os words **W0–W23**. Assim:

Ciclo	wr_en	wr_addr	Word	Campo
C24	1	168	W8	packet_count
C25	1	169	W9	packet_count
C26	1	170	W10	byte_count
C27	1	171	W11	byte_count
C28	1	172	W12	byte_count
C29	1	173	W13	byte_count
C30	1	179	W19	last_timestamp
C31	1	180	W20	last_timestamp
C32	1	181	W21	last_timestamp
C33	1	182	W22	last_timestamp
C34	1	183	W23	last_timestamp

A Flow Table confirma cada operação através de wr_done, conforme o protocolo definido.

5. Implementação

```

module flow_table #(
    parameter DATA_WIDTH = 16,
    parameter ADDR_WIDTH = 21,
    parameter MEM_DEPTH = 2**ADDR_WIDTH
)(
    input logic clk,

    // Read
    input logic rd_en,
    input logic [ADDR_WIDTH-1:0] rd_addr,
    output logic [DATA_WIDTH-1:0] rd_data,

```

```

output logic rd_valid,

// Write
input logic wr_en,
input logic [ADDR_WIDTH-1:0] wr_addr,
input logic [DATA_WIDTH-1:0] wr_data,
output logic wr_done
);

logic [DATA_WIDTH-1:0] mem [0:MEM_DEPTH-1];

always_ff @(posedge clk) begin

// Leitura
if (rd_en) begin
    rd_data <= mem[rd_addr];
    rd_valid <= 1'b1;
end
else begin
    rd_valid <= 1'b0;
end

// Escrita
if (wr_en) begin
    mem[wr_addr] <= wr_data;
    wr_done <= 1'b1;
end
else begin
    wr_done <= 1'b0;
end
end
end

```